

Stacks

DSA27 — Lecture 06 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 6 · Fall 2026

Where We Are

Two structures, one question

Week 4: the **dynamic array** — cheap at the **end**. Week 5: the **linked list** — cheap at the **front**.

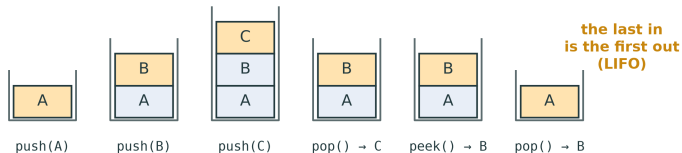
Today: an ADT that only ever touches **one end** — so either structure can keep its contract in $O(1)$. Which, and how?

1. The Stack ADT: push, pop, peek — LIFO
2. Two honest implementations, and one dishonest one
3. Balanced brackets
4. Postfix: expressions without parentheses
5. Infix to postfix: the shunting-yard algorithm
6. The stack you have used all term: the call stack

The Stack ADT

Last in, first out

A stack: push, pop and peek all act on the top — and only the top



Operation	Meaning	Cost
push(x)	put x on top	$O(1)$
pop()	remove and return the top — IndexError if empty	$O(1)$
peek()	return the top without removing it — IndexError if empty	$O(1)$
is_empty(), len(s)		$O(1)$

Two Honest Implementations

The top goes where the $O(1)$ end is

One ADT, two honest implementations — the top goes where the $O(1)$ end is

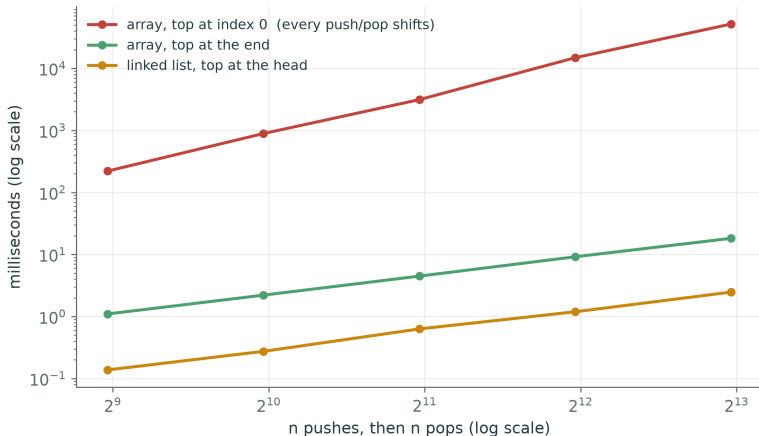


- On a **dynamic array**: top = the **end**. append and pop() — nothing moves.
- On a **linked list**: top = the **head**. push_front and pop_front.

...and one dishonest one

Put the top at **index 0** of an array, and **every** push and pop shifts the whole stack: $O(n)$ each.

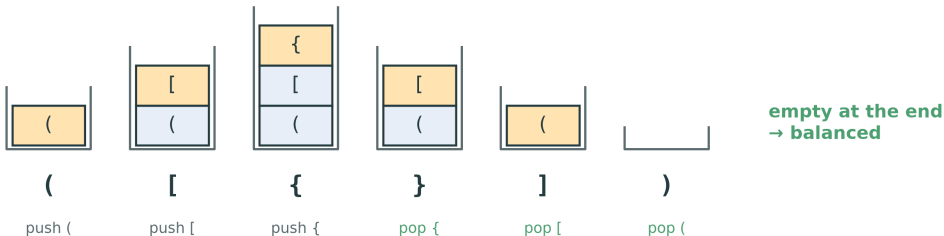
Same ADT, three designs: where you put the top decides $O(n)$ or $O(n^2)$



Application 1: Balanced Brackets

Every closer must match the most recent opener

`is_balanced("([{}])")`: push each opener; each closer must match the top



- **Opener** ([{ : push it.
- **Closer**)] } : the stack must be **non-empty**, and the popped opener must be its **partner**.
- **End of text**: the stack must be **empty**.
- Anything else: ignore.

Application 2: Postfix

Expressions with no parentheses

Infix (usual)	Postfix (reverse Polish)
$3 + 4$	$3\ 4\ +$
$3 + 4 * 2$	$3\ 4\ 2\ *\ +$
$(3 + 4) * 2$	$3\ 4\ +\ 2\ *$
$8 - 3 - 2$	$8\ 3\ -\ 2\ -$

In **postfix**, each operator comes **after** its operands. No parentheses, no precedence rules — the order says everything.

Evaluating postfix with one stack

- **Number:** push it.
- **Operator:** pop **two** values, apply, push the result.
- **End:** exactly one value is left — the answer.

Token	Action	Stack (top on the right)
3	push	3
4	push	3 4
2	push	3 4 2
*	pop 2 and 4, push 8	3 8
+	pop 8 and 3, push 11	11

Application 3: Infix to Postfix

The shunting-yard algorithm

Read the infix tokens left to right, with an **output list** and an **operator stack**:

1. **Number** → output.
2. **Operator** → first pop to the output every operator on the stack with **higher or equal precedence**; then push this one.
3. **(** → push.
4. **)** → pop to the output until the **(**; discard the **(**.
5. **End** → pop everything left to the output.

Precedence: * / above + -.

Trace: 3 + 4 * 2

Token	Output	Stack
3	3	
+	3	+
4	3 4	+
*	3 4	+ * $\leftarrow$ + is lower, so it stays
2	3 4 2	+ *
end	3 4 2 * +	

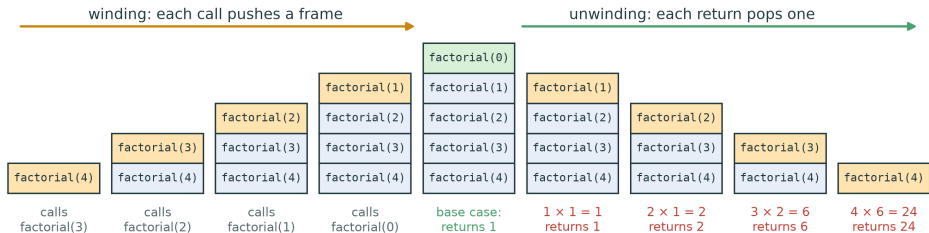
Trace: (3 + 4) * 2

Token	Output	Stack
(		(
3, +, 4	3 4	(+
)	3 4 +	← pop to (
*, 2	3 4 + 2	*
end	3 4 + 2 *	

Application 4: The Call Stack

You have been using a stack since Week 3

factorial(4) on the call stack — time runs left to right



Each call **pushes** a frame; each return **pops** one. Recursion **is** a stack.

This Week

Exercises: dsa/stack.py

Function	Target	The trap
<code>Stack.push(value)</code>	$O(1)$ amortised	top at the end of <code>self._items</code>
<code>Stack.pop()</code>	$O(1)$	<code>IndexError</code> when empty
<code>Stack.peek()</code>	$O(1)$	<code>IndexError</code> when empty; do not remove
<code>is_balanced(text)</code>	$O(n)$	three checks: partner, non-empty, empty at the end
<code>infix_to_postfix(tokens)</code>	$O(n)$	pop on higher or equal precedence; (and)

```
pytest tests/test_stack_queue.py -v -k "stack or balanced or postfix"
```

Your Stack stands on your DynamicArray: tests/test_dynamic_array.py must pass first.

Homework 6 — before Lecture 07

1. **Implement** `dsa/stack.py` until the stack tests pass.
2. **Trace** `infix_to_postfix` on $8 - 3 - 2$, $2 * 3 + 4$ and $(1 + 2) * (3 - 4) / 5$, as tables. Then evaluate each result by hand with the postfix algorithm.
3. **Write** `evaluate_postfix` in `dsa/translation.py` (it only needs your `Stack`), and check it against your traces.
4. **Design**. Sketch an undo/redo system for a text editor with two stacks. What happens to the redo stack when the user types after undoing?

Summary

Seven things to keep

1. A stack is **LIFO**: push, pop and peek act on the **top** only, all $O(1)$.
2. Empty pop and peek raise **IndexError**.
3. On an array the top is the **end**; on a linked list the **head**. Both honest.
4. Top at index 0 of an array: $O(n)$ per operation — same tests pass, wrong design.
5. **Balanced brackets**: push openers; each closer must match the top; empty at the end.
6. **Postfix**: push numbers; an operator pops two — **second popped is the left operand**.
7. **Shunting-yard**: operators wait on a stack and leave on **higher or equal** precedence, or at).

Week 7 — Queues. First in, first out: the ADT that needs **both** ends — and the week your `DynamicArray` finally meets an operation it is bad at.